

# Estimation of Human-perceived Difficulty of Chess Positions

Zala Urh

zu0476@student.uni-lj.si  
Faculty of Computer and  
Information Science,  
University of Ljubljana  
Večna pot 113  
SI-1000 Ljubljana, Slovenia

Matej Belšak

mb4390@student.uni-lj.si  
Faculty of Computer and  
Information Science,  
University of Ljubljana  
Večna pot 113  
SI-1000 Ljubljana, Slovenia

Luka Bajić

lb4129@student.uni-lj.si  
Faculty of Computer and  
Information Science,  
University of Ljubljana  
Večna pot 113  
SI-1000 Ljubljana, Slovenia

## ABSTRACT

This paper overviews various methods for predicting the difficulty of a given chess position from the viewpoint of human players. Feature engineering is implemented via limited-depth search with a chess engine, with the emphasis on interpretability. Modeling and evaluation are split into classification, regression and ranking.

## KEYWORDS

chess, classification, regression, ranking, feature engineering

## 1 INTRODUCTION

In this paper we compare various methods for determining the difficulty of a given chess position for human players, using chess engine evaluations and other attributes of the position. This problem can be defined in the following three ways:

- classification: the model classifies positions into pre-defined classes (e.g. easy/medium/hard), or binned ELO ratings.
- regression: the model directly predicts an estimated ELO rating of the position.
- ranking: the model orders a set of positions from easiest to hardest, or ranks a single position in relation to all positions in the training set.

Full implementation is available at <sup>1</sup>.

## 2 RELATED WORK

Recent approaches for estimating the difficulty of a chess position, e.g. [4] are largely based on the deep learning paradigm and the input of these models consists of some encoding of the chess position, but no information that would provide interpretability for human players. In this paper we therefore focus on finding features that achieve good classification score, but also provide a meaningful interpretation regarding the reasons why a position is considered difficult or easy for human players.

Our approaches are based on insights provided by [2] [1] [5]. These studies developed numerous attributes that can be obtained by performing a chess engine search of certain depth, starting in the initial position we wish to evaluate. The search is constrained in such a way that only moves which human players would typically assess as meaningful are explored, which aims to provide an estimate of how complex the search tree around the position is.

## 3 METHODOLOGY

To train and evaluate our models, we use a subset of the Lichess puzzle dataset <sup>2</sup> containing 1000 puzzles with ratings ranging between 399 and 3092. For classification we create classes "easy", "medium" and "hard" by splitting this interval equidistantly. The dataset is split into training set and test set with the ratio 80:20. Both sets are standardized to have zero mean and unit variance.

### 3.1 Evaluation Metrics

For classification problem we use the standard metrics: classification accuracy, F1 score and AUC. For regression we use mean absolute error (MAE) because unlike mean squared error, it has a more intuitive interpretation on this dataset. We also report which percentage of the rating interval this MAE covers. For ranking we use Normalized Discounted Cumulative Gain (NDCG) and Kendall  $\tau$  to determine the accuracy of the model's ranking against the ground truth, which is simply a ranking assigned by ordering the ratings in the dataset.

### 3.2 Attributes

Other than a list of themes for each puzzle, the original dataset contains no attributes that would allow modeling of perceived difficulty for human players, so we expand it with the following attributes:

- Stockfish parameters for each depth between 1-3 from starting position. These parameters include centipawn evaluations, number of searched nodes, WDL (win/draw/loss) and selective depth.
- material count and piece counts for each side.
- attributes from [5], computed for depth=3, excluding "Seemingly Good" due to ambiguous definition.
- binary attributes for solving success by Stockfish engine with different playing strengths.

### 3.3 Hierarchical Model

We propose a hierarchical classification/regression model, consisting of a main model, which classifies the position into one of the predetermined equidistant rating bins and a dedicated, more precise, separately trained model for each of the bins.

<sup>1</sup><https://github.com/bajicluka01/chess-puzzle-evaluation/tree/main>

<sup>2</sup><https://database.lichess.org/#puzzles>

## 4 RESULTS

### 4.1 Classification

Table 1 displays the performance of the standard machine learning models on a dataset with 1000 samples and all available attributes. Class distribution is 39.1% "easy", 38.9% "medium" and 22.0% "hard".

XGBoost uses 100 estimators and learning rate 0.1. SVM uses regularization parameter  $C = 1$  and balanced class weight. Random Forest uses 100 estimators. kNN uses 10 nearest neighbors. MLP uses  $layers = [2048, 1024, 512, 256, 128]$ , ReLU activation function, ADAM solver, regularization  $\alpha = 0.001$  and batch size 512. Default values are used for all remaining parameters. Ensemble uses a soft voting scheme on the outputs of all other models, meaning that probabilistic predictions are aggregated and finally converted into a class value.

Table 1: Classification scores on 1k dataset.

| model             | CA           | F1           | AUC          |
|-------------------|--------------|--------------|--------------|
| XGBoost           | 0.575        | 0.574        | 0.756        |
| kNN               | <b>0.635</b> | <b>0.621</b> | 0.773        |
| Random Forest     | 0.620        | 0.616        | 0.772        |
| Gradient Boosting | 0.575        | 0.563        | 0.758        |
| MLP               | 0.630        | <b>0.621</b> | 0.770        |
| SGD               | 0.550        | 0.551        | 0.693        |
| SVM               | 0.595        | 0.574        | 0.756        |
| Ensemble          | 0.615        | 0.612        | <b>0.779</b> |

In Table 2 we show the results of an ablation study aimed to determine the utility of each subset of attributes. We use the best performing model in terms of CA from the previous experiment: kNN with  $k = 10$ . Baseline refers to simple attributes, such as centipawn evaluations and number of searched nodes for each depth. These attributes are used in each experiment. Study refers to attributes from [5].

Table 2: Ablation study on 1k dataset.

| model             | CA    | F1    | AUC   |
|-------------------|-------|-------|-------|
| baseline          | 0.565 | 0.561 | 0.671 |
| counts            | 0.555 | 0.545 | 0.688 |
| study             | 0.565 | 0.548 | 0.716 |
| Stockfish         | 0.565 | 0.557 | 0.752 |
| themes            | 0.565 | 0.554 | 0.704 |
| counts, study     | 0.530 | 0.508 | 0.723 |
| counts, Stockfish | 0.575 | 0.563 | 0.716 |
| counts, themes    | 0.565 | 0.553 | 0.713 |
| study, Stockfish  | 0.580 | 0.561 | 0.766 |
| study, themes     | 0.58  | 0.570 | 0.717 |
| all attributes    | 0.635 | 0.621 | 0.773 |

This confirms the usefulness of attributes from [5] and also confirms the usefulness of themes. The contribution of solving success

is negligible, possibly due to an inherent flaw in the implementation of Stockfish playing strength: weaker play is simulated by limiting the depth of the search tree and imposing a time constraint on the search, however, this might not accurately simulate the puzzle-solving ability of a player with a certain ELO, rendering the attributes useless.

### 4.2 Regression

Regression results are shown in Table 3. XGBoost uses 100 estimators and learning rate 0.1. MLP uses  $layers = [256, 256, 128, 128, 128, 64]$ , ReLU activation function, adaptive learning rate, ADAM solver, regularization parameter  $\alpha = 0.001$  and batch size 256. Default values are used for all remaining parameters. Ensemble model performs unweighted averaging on the outputs of all other models.

Table 3: Regression scores on 1k dataset.

| model             | MAE            | % of interval |
|-------------------|----------------|---------------|
| XGBoost           | 329.836        | 12.2          |
| Linear regression | 321.600        | 11.9          |
| MLP               | 317.419        | 11.8          |
| Bayes Ridge       | 314.587        | 11.7          |
| Ensemble          | <b>310.385</b> | <b>11.5</b>   |

### 4.3 Ranking

Ranking results are shown in Table 4. XGBRanker uses 100 estimators and learning rate 0.1, CatBoostRanker uses 1000 iterations and learning rate 0.1. Default values are used for all remaining parameters.

Table 4: Ranker scores on 1k dataset.

| model          | NDCG        | $\tau$      |
|----------------|-------------|-------------|
| XGBRanker      | 0.84        | 0.41        |
| LGBMRanker     | <b>0.87</b> | 0.43        |
| CatBoostRanker | 0.85        | <b>0.48</b> |

Kendall  $\tau$  is a metric defined on  $[-1, 1]$ , so these scores indicate that the rankers do capture at least some notion of difficulty from the attributes.

## 5 CONCLUSIONS

Future work would consist of checking if these approaches are suitable also for strategic/nondescript positions, rather than just puzzles. Here, the main challenge is in assigning the ground truth ELO to these positions, since consensus among human players is more difficult to achieve.

Additionally, Maia chess engine [3] [6] could be used as an alternative to Stockfish, as it is designed to approximate human play more closely. However, we ran a few experiments on known positions, and again this similarity to human-like play does not directly translate to human-like puzzle solving, so additional model training may be required, which is outside the scope of this paper.

## REFERENCES

- [1] Ivan Bratko, Dayana Hristova, and Matej Guid. 2016. *Search Versus Knowledge in Human Problem Solving: A Case Study in Chess*. Vol. 27. 569–583. [https://doi.org/10.1007/978-3-319-38983-7\\_32](https://doi.org/10.1007/978-3-319-38983-7_32)
- [2] Ivan Bratko, Dayana Hristova, and Matej Guid. 2021. Predicting Problem Difficulty in Chess. In *Human-Like Machine Intelligence*, Stephen Muggleton and Nicholas Chater (Eds.). Oxford University Press. <https://doi.org/10.1093/oso/9780198862536.003.0024> arXiv:<https://academic.oup.com/book/0/chapter/350718010/chapter-pdf/43431216/oso-9780198862536-chapter-24.pdf>
- [3] Reid McIlroy-Young, Siddhartha Sen, Jon Kleinberg, and Ashton Anderson. 2020. Aligning Superhuman AI with Human Behavior: Chess as a Model System. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '20)*. ACM, 1677–1687. <https://doi.org/10.1145/3394486.3403219>
- [4] Szymon Miłosz and Paweł Kapusta. 2024. Predicting Chess Puzzle Difficulty with Transformers. arXiv:2410.11078 [cs.LG] <https://arxiv.org/abs/2410.11078>
- [5] Simon Stoiljković, Ivan Bratko, and Matej Guid. 2015. A Computational Model for Estimating the Difficulty of Chess Problems. <https://api.semanticscholar.org/CorpusID:204857616>
- [6] Zhenwei Tang, Difan Jiao, Reid McIlroy-Young, Jon Kleinberg, Siddhartha Sen, and Ashton Anderson. 2024. Maia-2: A Unified Model for Human-AI Alignment in Chess. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=XWlkhRn14K>